

DGvGAN: Dynamic Malware Analysis

Comparing **Deep Graph Convolutional Neural Networks** and **Generative Adversarial Networks** for modern malware detection through behavioural analysis

Supervisor: (Prof.) Dr. Devesh C. Jinwala

Team:

Ameya Srivastava (U22CS102),
Auritra Dey (U22CS127),
Bittu Kumar (U22CS130),
Pavitra Kumar (U22CS121)

I.D.: Team #1





Introduction – Detecting Modern Malware

Traditional Static Analysis

Converting **binaries** to **grayscale images** for classification is computationally expensive and **loses crucial runtime behaviour information**.

Modern Threats

Polymorphic malware uses obfuscation techniques that change code signatures, making traditional static detection methods unreliable.

Modern Approach: Dynamic Analysis

Analysing API call sequences captures actual program behaviour, providing resilience against many modern evasion tactics.

Existing Methods – Traditional Static Analysis

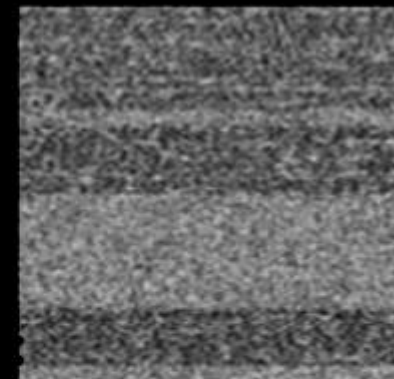
Image Translation & CNN-based Classification

- **Training:** Convert malware executable binaries (.exe) into grayscale images after preprocessing, label each as **benign (0)** or **malware (1)**. Train a **CNN** for **supervised image-based binary classification**.
- **Deployment:** An executable is uploaded, preprocessed and fit into a large CNN for classification.



Malware
Executable

Preprocessing



Grayscale Image
(malware: 1)

Why Static Analysis Fails

→ High Computational Cost

Reading and preprocessing large binaries into images introduces **significant latency** in deployment.

→ Fragile Detection

Minor binary changes drastically alter images, making detection unreliable against modern, evasive threats.

→ Information Loss

Flattening code into pixels **discards key runtime behaviours (almost ~99%)** and control-flow relationships.

→ Memory Intensive

Large CNNs and intermediate images **increase processing requirements**, making the methods impractical for enterprise use.

Issues:

- Commonly downsampling binaries (**e.g. 10MB**) to small images (e.g. **256 x 256**) discards **~99% of data bytes***, losing critical low-level malware cues.
- Using full-resolution images is impractical for large-scale inference due to high memory & compute demands.

* $10 \text{ MB} = 10 \times 2^{20}$, $256 \times 256 = 2^{16}$, $Loss = \frac{10 \times 2^{20} - 2^{16}}{10 \times 2^{20}} = \frac{159}{160} \equiv 99.375\%$

Existing Methods – Dynamic Analysis

Deep Learning Models

- **Sequence models** like **RNNs** and **LSTMs** are widely used to model temporal sequence API calls for dynamic malware analysis
- **Graph models** like **DGCNNs** (Oliveira & Sassi 2019) show superior performance to sequence models

BUT...

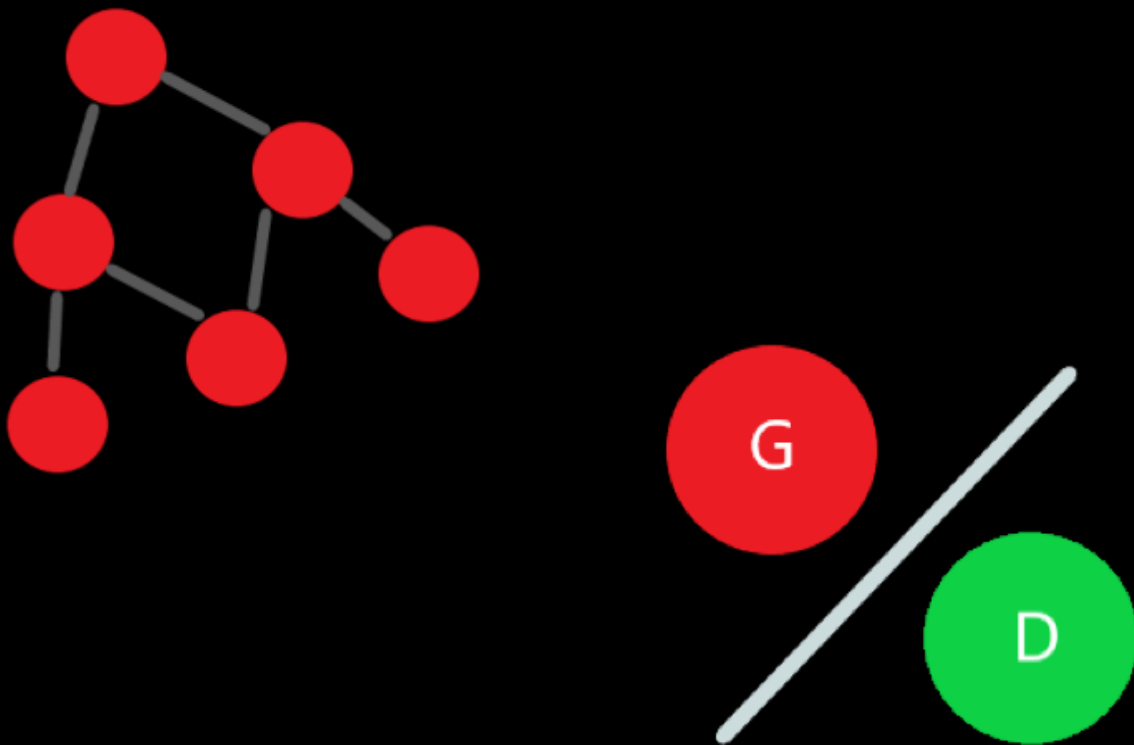
- GAN's, that are **generative models**, are understudied for this task – this provides a good research opportunity

Motivation

- Static analysis techniques **are outdated** for modern malware attack vectors
- Of all deep learning models, generative architectures like **GANs are understudied** for malware detection
- Graph models **are promising** – comparing them with generative models will be beneficial

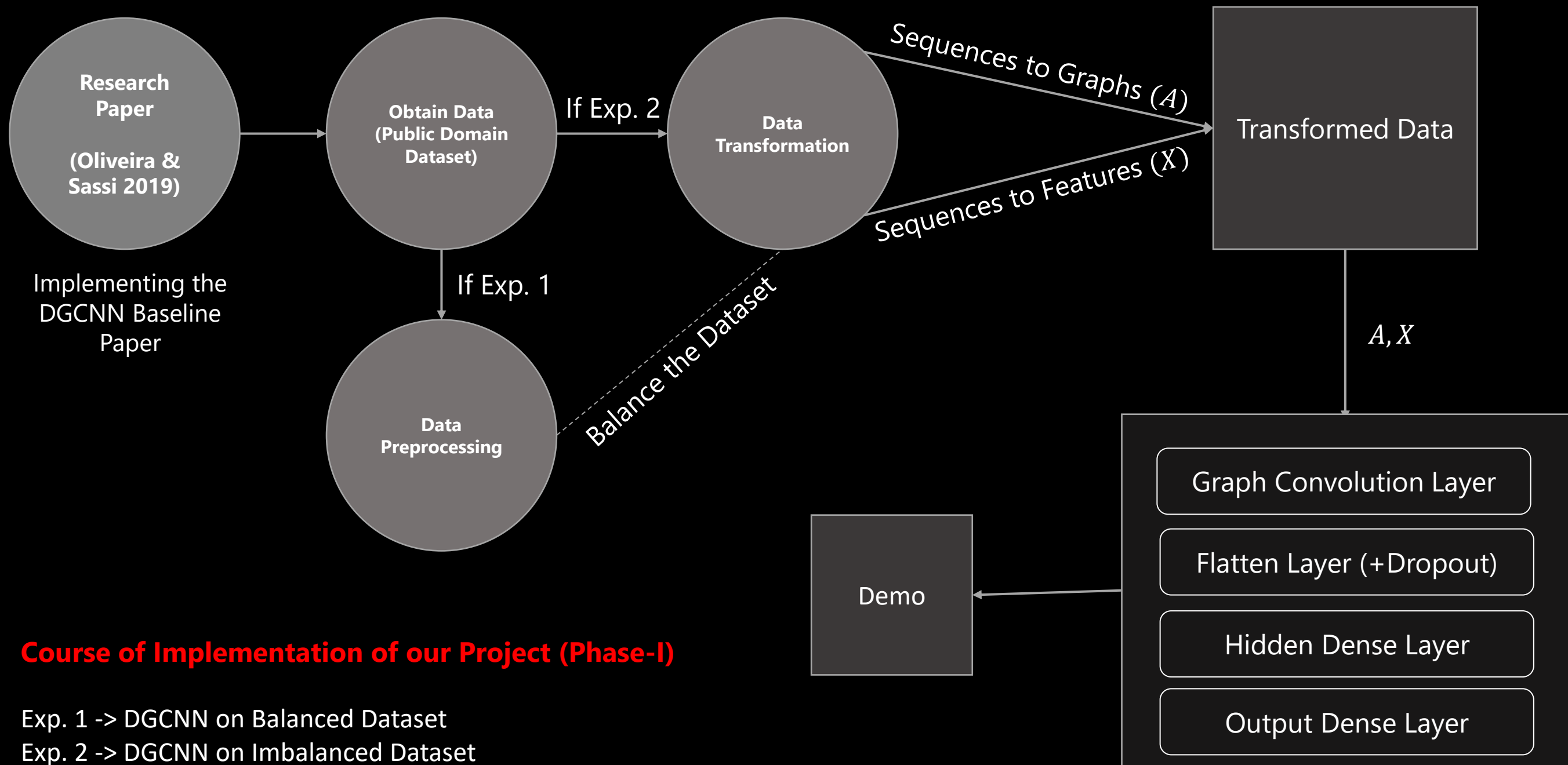
Project Abstract

Given the **criticism** that the common static methods face, we decided to explore the **dynamic analysis** of **malware** as **anomaly detection with GANs** and compare it with the **Deep Graph CNN (DGCNN)** as a baseline.



Our Objectives

- **Phase I (✓):** Build a baseline **DGCNN** to model API call sequences as behavioural graphs, outperforming LSTMs. (**Oliveira & Sassi, 2019**)^[1]
- **Phase II:** Develop a **GAN** to learn goodware API call distributions and distinguish real from fake/anomalous sequences.
- **Compare** both **dynamic analysis** methods to evaluate effectiveness.



Course of Implementation of our Project (Phase-I)

Exp. 1 -> DGCNN on Balanced Dataset

Exp. 2 -> DGCNN on Imbalanced Dataset

Note:

- Our dataset contains **Windows API call sequences**, one row each per executable file
- It's **highly imbalanced**, with 42,797 malware and 1,097 benign sequence samples

Model-1, DGCNN

Before We Begin...

Why specifically the DGCNN?

- It's a **promising**, novel method that demonstrates the use of **graph** models with performance superior to **sequence models**
- Instead of implementing the common sequence models, we choose to implement and test two new paradigms (the DGCNN and our GAN) together

Why implement a GAN for Anomaly Detection? Why not simpler models or VAE's?

- Simpler classification models focus on **maximizing class separation**, but we need a model that **learns the probability distribution** of normal goodware behaviour instead
- A GAN's **discriminator learns a sharp decision boundary** of what is authentic, providing more precise anomaly signals than **VAEs that don't have discriminators**
- GAN's are **understudied models** for this type of task, providing us the opportunity to research and compare their use for the same

Phase I: Baseline DGCNN Implementation

- We followed **A.S. Oliveira & R. J. Sassi's** paper, "**Behavioral Malware Detection Using Deep Graph Convolutional Neural Networks**" (2019) to implement the **DGCNN** baseline as our **first** step.
- The authors collected **40,000+** benign and malicious samples of **Windows Process API call sequences** from controlled experiments in **Cuckoo Sandbox**.
- They implemented the **Deep Graph Convolutional Neural Network (DGCNN)** on both balanced and imbalanced datasets.
- Our goal was to implement the **DGCNN** as a comparison baseline for our **GAN** (yet to implement), following their methods in **Experiment 1 (Balanced Data)**, and **Experiment 2 (Imbalanced Data)** of their paper.

Dataset Overview

42,797

Malware Samples

Comprehensive collection
from Cuckoo Sandbox
environment

1,079

Goodware Samples

Benign software for
comparison and training

307

Unique API Calls

Global vocabulary of distinct
system calls

100

Calls Per Sample

Maximum recorded API
sequences per program

NOTE: The dataset exhibits **severe class imbalance**, reflecting real-world malware distribution patterns.

- We implemented **Model-1** (Baseline) on **two** forms of the collected data:
 - **Experiment 1**, Balanced using **Random-Under Sampling (RUS)** of the majority malware class for a 50-50 class balance. There are hence a total of 2,158 samples.
 - **Experiment 2**, On the original data distribution in the dataset itself, of 43,877 samples.
- A train-test split of **80/20** was used in both experiments.
- Both experiments underwent **common data transformation** processes to make them fit into the DGCNN's shape requirements.

API Call Vocabulary

ID	Windows API Calls
0	NtOpenThread
1	ExitWindowsEx
2	FindResourceW
...	...
150	GetSystemInfo
...	...
306	__exception__

Dataset Sample

hash	t_1	t_2	t_3	...	t_100	malware
071e8c3j3..	124	112	301		123	1
33f8e8yu4..	0	274	83		222	0
563qe7id6..	200	150	306		245	1
87a32hk92..	59	151	242		40	0

Data Preprocessing & Transformation

For **Experiment 1**, the data imbalance was handled by performing **Random Undersampling (RUS)** on the **majority** malware class, so that both of the classes could have a **50-50 balance**.

Apart from handling the imbalance, we had to perform the following data transformations to make the data workable with the DGCNN (according to the paper):

01 | Sequence to Graph Representation (Seq2Graph)

Transform API call sequences (rows) into **behavioural graphs represented as adjacency matrices** (A) where nodes represent calls and edges show temporal succession during execution.

02 | Feature Matrix Creation

One-hot encode API sequences into 307×100 matrices (X), transposed for graph convolution compatibility (matrix multiplication).

Sample API Sequence (Top 100 Calls)

t_0	t_1	t_2	t_3	t_4	t_5	t_6	t_7	...
112	274	158	215	274	158	215	298	...

43877 rows, 100 columns

Adjacency Matrix (A)

From/To	112	158	215	274	298
112	0	0	0	1	0
158	0	0	1	0	0
215	0	0	0	1	1
274	0	1	0	0	0
298	0	0	0	0	0

Shape: 307×307

Node Feature Matrix (X)

API Call ID	t_0	t_1	t_2	t_3	t_4	t_5	t_6	t_7
112	1	0	0	0	0	0	0	0
158	0	0	1	0	0	1	0	0
215	0	0	0	1	0	0	1	0
274	0	1	0	0	1	0	0	0
298	0	0	0	0	0	0	0	1

Shape: 307×100

Data Transformation Steps

API Sequence to Graph Adjacency Matrix (A)	API Sequence to Node Feature Matrix (X)
$n \leftarrow \text{len}(\text{sequences})$ $N \leftarrow 307$ initialize: A (n, N, N) with 0's for i in range(n): $\text{seq} \leftarrow \text{sequences}[i]$ for j in range($\text{len}(\text{seq}) - 1$): $\text{curr}, \text{next} \leftarrow \text{seq}[j], \text{seq}[j + 1]$ if $0 \leq \text{curr} \leq N$ and $0 \leq \text{next} \leq N$, then $A[i, \text{curr}, \text{next}] \leftarrow 1$	$n \leftarrow \text{len}(\text{sequences})$ $N \leftarrow 307$ $s \leftarrow 100$ initialize: X (n, N, s) with 0's for i in range(n): $\text{seq} \leftarrow \text{sequences}[i]$ for j in range($\text{len}(\text{seq})$): $\text{id} \leftarrow \text{seq}[j]$ if $0 \leq \text{id} < N$, then $X[i, \text{id}, j] \leftarrow 1$
Turns the API call sequences (rows) to adjacency matrix-based graphs. n is the length of <i>sequences</i> (43,877) N is the total number of distinct API calls (307)	Creates a node-feature matrix based on the API call sequence samples (rows). s is the length of a sequence (100), that is fixed in the dataset.

DGCNN (Model-1) Implementation

Our model implementation replicates **Model-1 (DGCNN)** from the paper, that has the following structure:

- A single **Graph Convolution Layer** based on the paper's implementation
- A **ReLU (Rectified Linear-Unit)** activation function
- A **Dropout** layer with a rate of **0.60** for regularization
- A **Flatten** layer followed by **Dense** layers for final binary classification

Key Considerations:

- The model was trained for at most **20 epochs**, and **early stopping** was implemented to prevent **overfitting**, since the model had a very large number of parameters (**1,260,929**). This also justifies the rather aggressive dropout rate of **0.60**, forcing the model to learn patterns that generalize to unseen data.
- An **Adam** optimizer with a learning rate of $\eta = 0.001$ was used with **Binary Cross Entropy Loss** for optimization, which are standard choices in **binary classification** tasks.
- The model was built and trained using **TensorFlow & Keras (2.15.0)**

Graph Convolution Layer

The core mathematical operation that enables learning from graph structures:

$$Z = f(\tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}} \cdot XW)$$

Where,

- X is the **node feature matrix**
- W is the **weight matrix** of the model
- $\tilde{A}$ is the **augmented adjacency matrix** $\tilde{A} = A + I$
- $\tilde{D}^{-1}$ is the inverse of the **degree matrix** of $\tilde{A}$

NOTE:

This is the general formula of graph convolution in a GCN formulated by **Kipf & Welling (2016)**. Our reference paper used **row normalization**, but we chose to use **symmetric normalization** instead, since it is often **recommended** for stable training and better generalization.

Cascading from Previous Steps:

- **Node feature matrices** (X) and **adjacency matrices** (A) were created in the **data preprocessing and transformation** stage, and they were fit into the **DGCNN** model as input.
- The original reference paper's^[1] **Algorithm 2** was directly implemented as code for this graph convolution layer and the larger model.

Graph Convolution Algorithm

Graph Convolution Algorithm	Comments
$\begin{aligned} & \text{Input: } A, X, W \\ & I \leftarrow \text{Identity}(A.\text{shape}) \\ & \tilde{A} \leftarrow A + I \\ & \tilde{D}_{ii} \leftarrow \sum_j \tilde{A}_{ij} \\ & \tilde{D}^{-1/2} \leftarrow \text{Diagonal}(\sqrt{\tilde{D}^{-1}}) \\ & \tilde{A}_{norm} \leftarrow \tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}} \\ & O \leftarrow \tilde{A}_{norm} X W \\ & H = \sigma(O) \end{aligned}$	• Input the Adjacency Matrix (A), Feature Matrix (X), Weight Matrix (W) • Create an Identity matrix (I) for self-loops of API calls • Augment ($\tilde{A}$) the Adjacency Matrix (A) with self-loops • Create a diagonal degree matrix ($\tilde{D}$) by summing each row j of $\tilde{A}$ to get the degree of each node • Create a diagonal matrix from the inverse square root of $\tilde{D}$, this will be used in the symmetric normalization • Calculate the normalized matrix $\tilde{A}_{norm}$ (Kipf & Welling, 2016)^[2] • Aggregate the features and apply weight transformation • Apply an activation function σ (ReLU) on the output O for final aggregation

References: Oliveira & Sassi (2019)^[1], Kipf & Welling (2016)^[2]

This algorithm has been implemented as the convolution layer in our DGCNN model

Forward Propagation

Layer Propagation	Comments
$Z \leftarrow \text{GraphConv}(A, X, W_{\text{conv}})$ $H \leftarrow \text{ReLU}(Z)$ $H_{\text{drop}} \leftarrow \text{Dropout}(H, \text{rate} = 0.60)$ $v \leftarrow \text{Flatten}(H_{\text{drop}})$ $v_{\text{dense}} \leftarrow \text{ReLU}(\text{Dense}(v, W_{\text{dense1}}))$ $O \leftarrow \sigma(\text{Dense}(v_{\text{dense}}, W_{\text{out}}))$ return O	• Pass A, X, W_{conv} to the <i>GraphConv</i> algorithm from the previous slide • Apply ReLU activation on the convolution output • Dropout nodes from the output with 60% probability to regularize • Flatten the convoluted output into a single vector • Pass the flattened single vector output to the Dense layers in the classification head to return raw logits, apply ReLU (standard) • Pass the output logits through the sigmoid activation to return a probability score between 0 and 1 (binary classification)

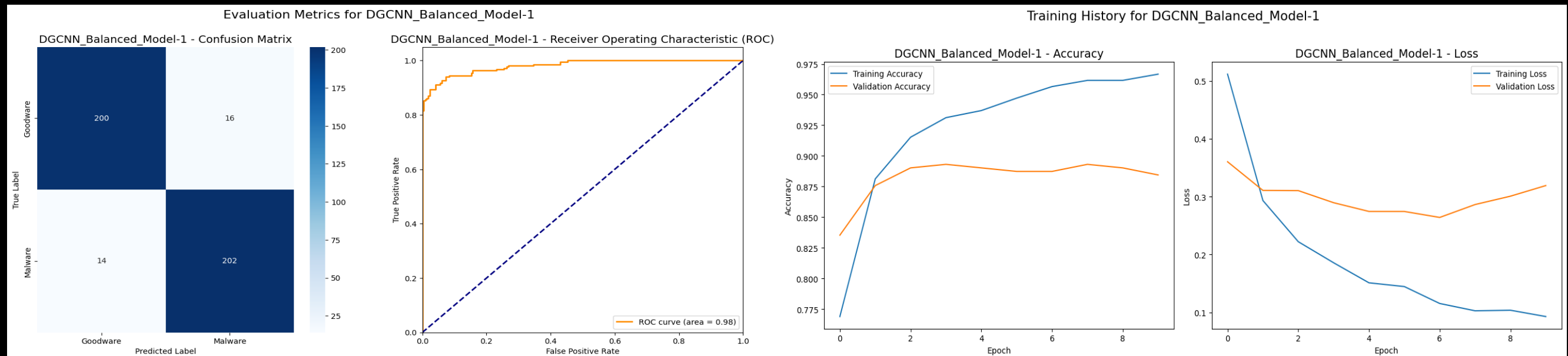
- It is assumed that X is transposed to be compatible for matrix multiplication (307×100)
- A dropout layer is implemented to aggressively prevent overfitting since the model is dense with **≈ 1.2 Million** learnable parameters, hence the high dropout rate of **60%**
- On a high level, the model contains 4 layers – the *GraphConv*, the *Flatten*, one hidden *Dense* and another output *Dense* layer (that make the classifier head).

DGCNN Evaluation on Experiment 1

- Our baseline **DGCNN** model reaches comparable, if not better performance than the reference paper's implementation.
- It correctly recalls **93.5% of Goodware (200/214)**, and **92.7% of Malware (202/218)**. It achieves an overall **F1-score of 93%** on the validation dataset of **432 samples**.
- An **AUC-ROC** score of **0.98** is achieved, indicating **great** classification performance.
- Some **mild** overfitting tendencies can be noted in the training history around **epoch 6**, but that is prevented from inflating further through **early-stopping**.

Balanced Dataset (Experiment 1)

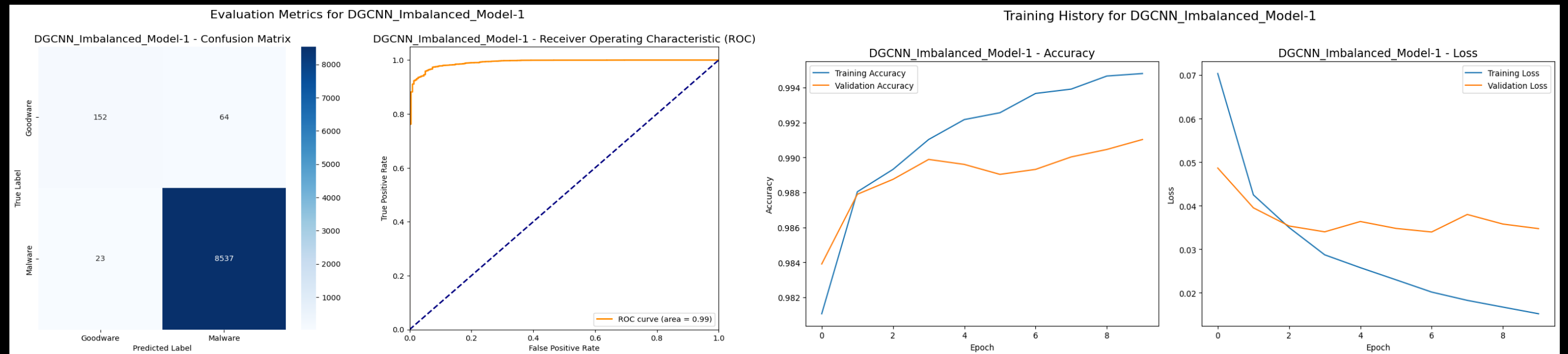
Metric	Our Implementation	Paper's Baseline
Accuracy	0.930	0.910
AUC-ROC	0.966 (best)	0.968 (best)
Precision	0.930	0.887
Recall	0.930	0.928



DGCNN Evaluation on Experiment 2

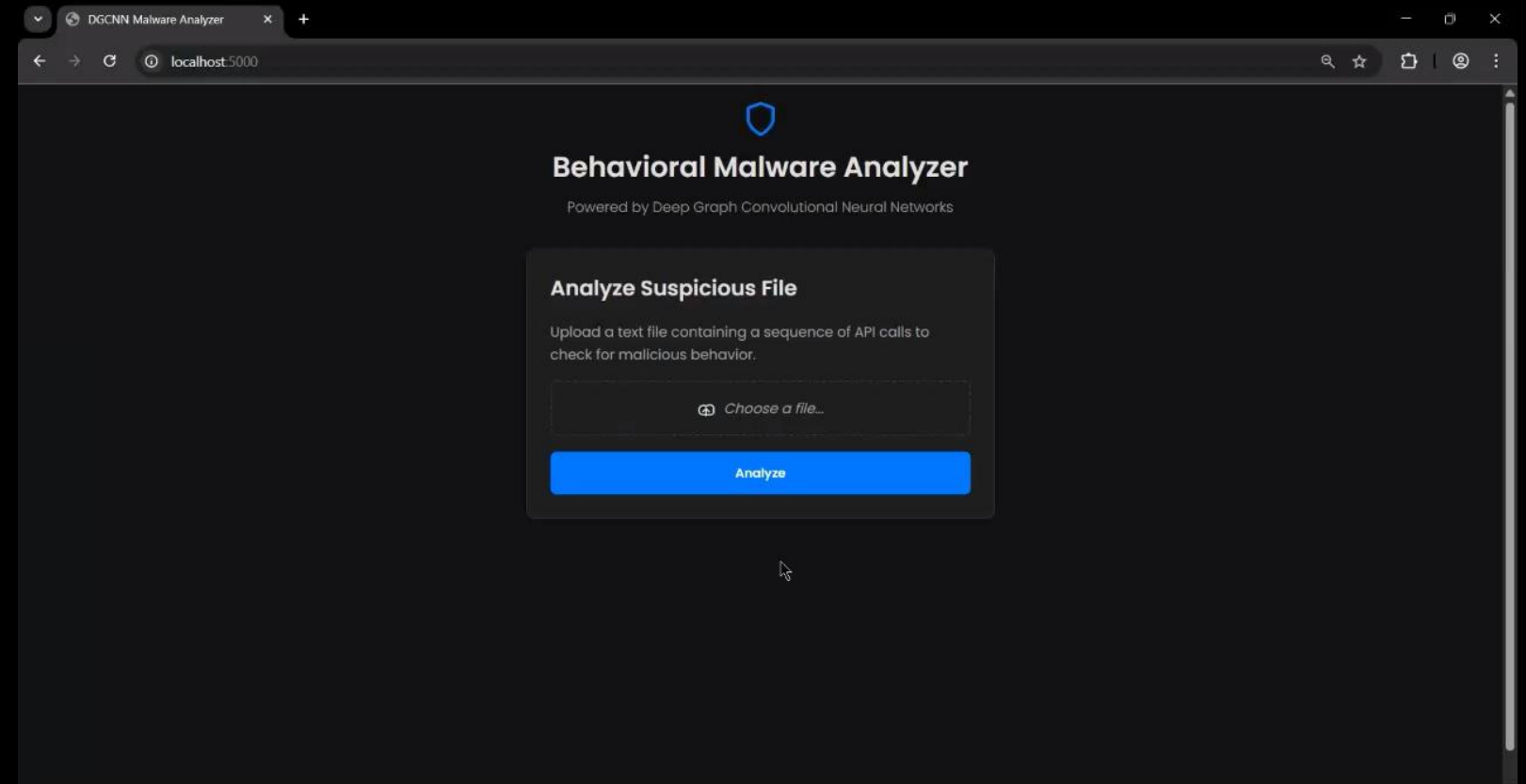
- Our model achieves an **ROC-AUC** of 0.983, but this cannot be relied on since it happens to be inflated for highly imbalanced datasets.
- We report our **macro-average F1-score** to be **0.890**, which is a more balanced measure of performance than the paper's reported overall F1-score of **0.994**.
- Despite the **40:1** imbalance, we report a **fairly** robust generalization performance.

Imbalanced Dataset (Experiment 2)		
Metric	Our Implementation	Paper's Baseline
AUC-ROC	0.983 (best)	0.968 (best)
F1-Score	0.890 (macro-avg)	0.994 (overall)



Working Demo

- For safety purposes, we can't actually download malicious .exe files (they're not easily available too)
- Instead, we decided to simulate a working demo of the model using .txt files that contain a sequence of Windows API calls – that would otherwise be obtained in a sandboxed environment
- To demonstrate, we take a few samples from our validation data and upload it to the model's interface for prediction



Phase II: Planned GAN Approach

With a robust baseline DGCNN implemented in **Phase I**, for the next section of our project - we plan to develop our proposed **GAN (Generative Adversarial Network)** model for analyzing malware as an anomaly detection task deviating from normal API sequence call behavior.

01 | Generator Training

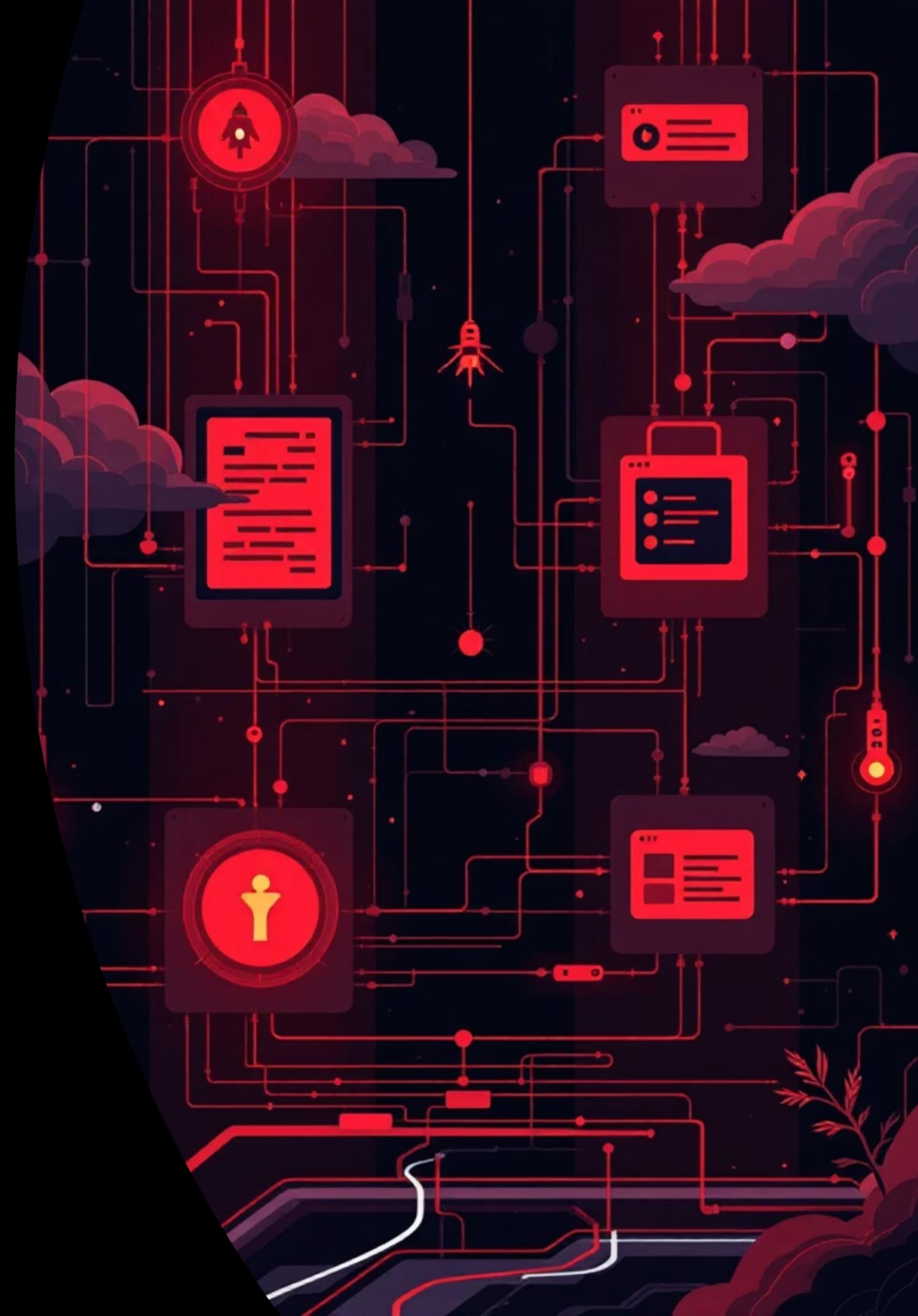
Learn the distribution of normal API call sequences from the 1,079 benign samples only, producing realistic goodware sequence samples.

02 | Discriminator Learning

Distinguish between real goodware sequences and generator-produced fakes/anomalies, learning the actual distribution of goodware.

03 | Anomaly Detection

Use the trained discriminator to score new sequences - with high scores for normal, low scores flag malware anomalies.



Final Objectives & Impact

Malware as Anomaly

Study if treating **malware** as anomalous from normal behaviour is effective for dynamic analysis based solutions that have been deployed.

Paradigm Comparison

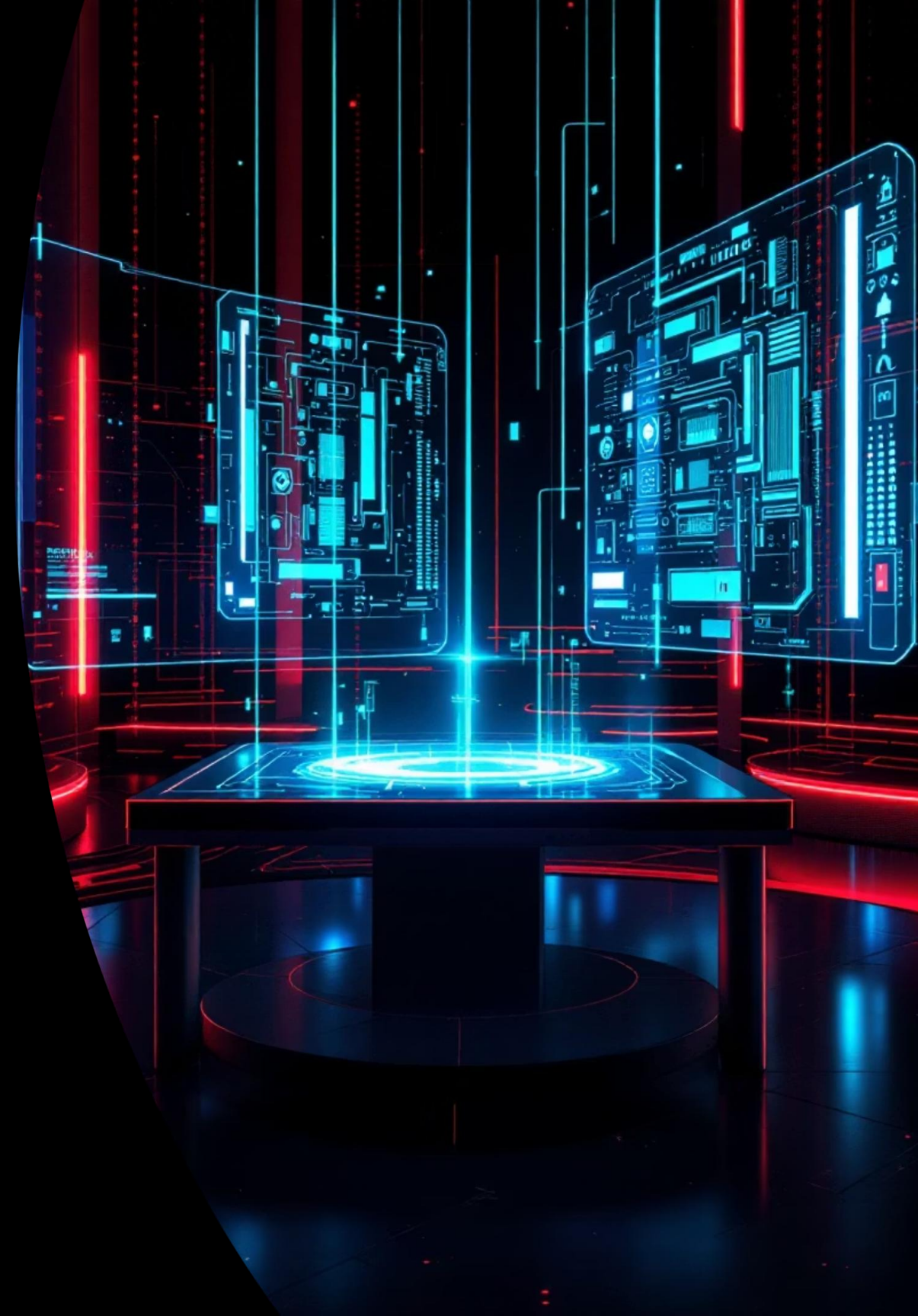
Perform a comprehensive comparison between the established graph-based approach and a new way of analyzing malware (i.e. the GAN)

Future Research

Fuel future research for the comparison of traditional malware analysis approaches with new, dynamic anomaly-based detection ones.

Applications

Deploy the GAN-based anomaly detector in real-world environments like SOC's or cloud-based scanner tools, powering Next-Gen Antivirus Engines (N-GAVs).



THANK YOU